Students Information:

Full Name: Konstantin Permin, Sabina Sadykova

Student ID: 10624223, 10624226

Group: ISU-US-24-2-a

Course Name: Database Management Systems

Date of Submission: May 12, 2026

Implementation of a Hospital Management System using PostgreSQL and MongoDB

Study Explanation

Objective

The primary goal of this study is to design and implement a robust "Hospital Management System" that automates healthcare workflows such as patient registration, doctor scheduling, surgical history tracking, and financial billing. The project demonstrates the practical application of a hybrid database approach: using a **Relational Database (PostgreSQL)** for transactional integrity and a **NoSQL Database (MongoDB)** for flexible, high-performance data retrieval.

Design and Implementation Notes

Relational Design (PostgreSQL):

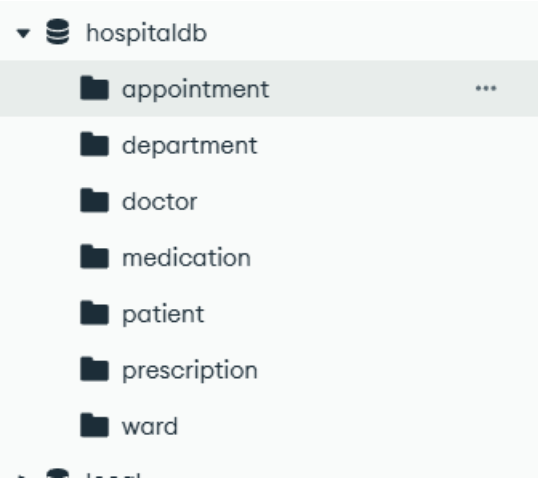
The database is built in the **3rd Normal Form (3NF)** to eliminate data redundancy. It consists of 11 interconnected tables:

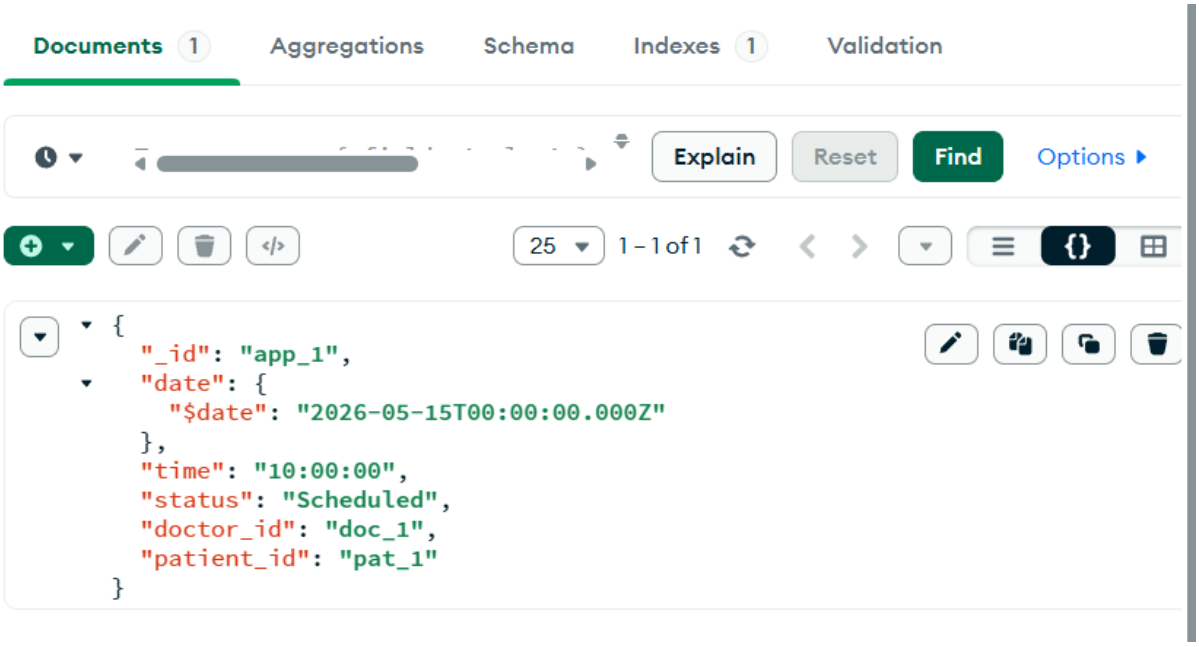
- **Staff Management:** Department, Specialization, Doctor.
- **Patient Care:** Patient, Ward, Hospitalization.
- **Clinical Records:** Appointment, Surgery, Medication, Prescription.
- **Finance:** Bill.

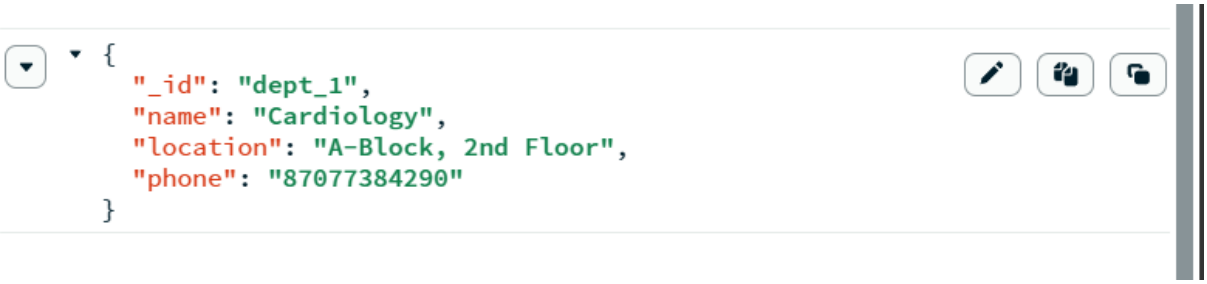
NoSQL Design (MongoDB):

For the non-relational part, we utilized an **Embedding Strategy** where surgery records and bills are stored directly within the Patient document. This design choice optimizes the system for "Read-Heavy" operations, such as viewing a patient's complete medical history in a single

query. I also implemented **JSON Schema Validation** to enforce data types and **Unique Indexes** on the email and phone fields to ensure data integrity.







```
{
  "_id": "doc_1",
  "firstName": "John",
  "lastName": "Smith",
  "phone": "87077384293",
  "email": "j@gmail.com",
  "specialization": {
    "title": "Cardiologist",
    "description": "Heart health expert"
  },
  "department_id": "dept_1"
}
```

```
{
  "_id": "med_1",
  "name": "Amoxicillin",
  "brand": "Amoxil",
  "description": "Antibiotic used to treat bacterial infections"
}
```

```
_id: "pat_1"
firstName : "Konstantin"
lastName : "Permin"
dateOfBirth : 2007-05-29T00:00:00.000+00:00
gender : "M"
address : "Petropavl, Pushkina 82a"
phone : "87077384292"
bills : Array (1)
  0: Object
    bill_id : "b_1"
    dateIssued : 2026-05-09T00:00:00.000+00:00
    totalAmount : 150.00
    status : "Pending"
surgeries : Array (1)
  0: Object
    surgery_id : "s_1"
    doctor_id : "doc_1"
    type : "General"
    outcome : "Successful"
    date : 2026-02-09T00:00:00.000+00:00
email : "k@gmail.com"
```

```
{
  "_id": "presc_1",
  "patient_id": "pat_1",
  "doctor_id": "doc_1",
  "medication_id": "med_1",
  "dosage": "500mg",
  "frequency": "Once daily",
  "issueDate": {
    "$date": "2026-05-09T00:00:00.000Z"
  }
}
```

```
{
  "_id": "w_1",
  "wardNumber": "101A",
  "wardType": "General",
  "capacity": 4,
  "occupancy": [
    {
      "patient_id": "pat_1",
      "checkInDate": {
        "$date": "2026-04-10T00:00:00.000Z"
      },
      "checkOutDate": {
        "$date": "2026-04-15T00:00:00.000Z"
      }
    }
  ]
}
```

Validation Example:

```
{
  $jsonSchema: {
    bsonType: 'object',
    required: [
      '_id',
      'firstName',
      'lastName',
      'dateOfBirth',
      'gender',
      'phone',
      'email'
    ],
    properties: {
      _id: {
        bsonType: 'string'
      },
      firstName: {
        bsonType: 'string',
        minLength: 2
      },
      lastName: {
        bsonType: 'string',
        minLength: 2
      },
      gender: {
        'enum': [
          'M',
          'F',
          'Other'
        ]
      },
      dateOfBirth: {
        bsonType: 'date'
      },
      phone: {
        bsonType: 'string',
        pattern: '^\\+?[0-9]{10,15}$'
      }
      email: {
        bsonType: 'string',
        pattern: '^.+@.+\\..+$'
      },
      bills: {
        bsonType: 'array',
        items: {
          bsonType: 'object',
          required: [
            'bill_id',
            'dateIssued',
            'status',
            'totalAmount'
          ],
          properties: {
            status: {
              'enum': [
                'Pending',
                'Paid',
                'Cancelled'
              ]
            },
            totalAmount: {
              bsonType: 'decimal'
            }
          }
        }
      },
      surgeries: {
        bsonType: 'array',
        items: {
          bsonType: 'object',
          required: [
            'surgery_id',
            'doctor_id',
            'type',
            'date'
          ],
          properties: {
            outcome: {
              'enum': [
                'Successful',
                'Complications',
                'Failed',
                'In Progress'
              ]
            }
          }
        }
      }
    }
  }
}
```

Business Rules

The following rules describe the core requirements of the system.

Departments and Staff

1. The hospital is divided into several **Departments** (e.g., Cardiology, Surgery, Pediatrics).
    - Each Department has: **DepartmentID** (unique), **Name**, **Location**, and **Phone**.
2. Each **Doctor** is assigned to exactly one Department.
3. A Department can employ many Doctors.
4. Each Doctor has:
    - **DoctorID** (unique)
    - FirstName, LastName
    - **SpecializationID** (FK)
    - **DepartmentID** (FK)
    - Phone, Email
5. **Specializations** are managed separately to standardize roles.
    - Each Specialization has: **SpecID** (unique), **Title**, and **Description**.

Patients and Hospitalization

1. A **Patient** is a person who receives medical care.
    - Each Patient has:
        - **PatientID** (unique)
        - FirstName, LastName
        - DateOfBirth, Gender
        - Address, Phone

2. Patients may be admitted to a **Ward** (room) during their treatment.
3. Each **Ward** has:
    - **WardID** (unique)
    - WardNumber, WardType (General, VIP, ICU)
    - Capacity (number of beds)
4. A **Hospitalization** record tracks the patient's stay:
    - Has a unique **HospID**.
    - Includes CheckInDate and CheckOutDate.
    - Is associated with exactly one Patient and one Ward.
    - A Patient may have many Hospitalizations over time.

Appointments and Treatments

1. An **Appointment** represents a scheduled meeting between a Patient and a Doctor.
    - Has a unique **AppointmentID**.
    - Includes AppointmentDate, Time, and Status (Scheduled, Completed, Cancelled).
    - Is associated with exactly one Patient and exactly one Doctor.
2. A Patient may have many Appointments; a Doctor may see many Patients.

Surgeries (Operations History)

1. The system tracks the history of surgical interventions.
2. Each **Surgery** has:

- **SurgeryID** (unique)
- SurgeryType (e.g., Appendectomy)
- SurgeryDate
- Outcome (Successful, Follow-up Required)

3. Each Surgery is performed on exactly one Patient and is led by exactly one Surgeon (Doctor).
4. A Patient may have zero or many Surgeries in their history.

Medications and Prescriptions

1. **Medications** are drugs available in the hospital pharmacy.
   - Each Medication has: **MedID** (unique), Name, Brand, and Description.
2. A **Prescription** is issued by a Doctor to a Patient for a specific Medication.
   - Has a unique **PrescriptionID**.
   - Includes Dosage, Frequency, and IssueDate.
   - Links exactly one Doctor, one Patient, and one Medication.

Billing and Payments

1. A **Bill** is generated for services provided to a Patient.
   - Has a unique **BillID**.
   - Includes DateIssued, TotalAmount, and PaymentStatus (Paid, Pending).
2. Each Bill is associated with exactly one Patient.
3. A Patient may have multiple Bills (e.g., one for an appointment, one for surgery).

Relational Schema

1. Department (id **INT PK**, name **VARCHAR(100)**, location **VARCHAR(255)**, phone **VARCHAR(20)**)

2. Specialization (id **INT PK**, title **VARCHAR(100)**, description **TEXT**)

3. Doctor (id **INT PK**, firstName **VARCHAR(50)**, lastName **VARCHAR(50)**, specializationID **INT FK → Specialization(id)**, departmentID **INT FK → Department(id)**, phone **VARCHAR(20)**, email **VARCHAR(255)**)

4. Patient (id **INT PK**, firstName **VARCHAR(50)**, lastName **VARCHAR(50)**, dateOfBirth **DATE**, gender **CHAR(1)**, address **TEXT**, phone **VARCHAR(20)**), email **VARCHAR(255)**)

5. Appointment (id **INT PK**, appointmentDate **DATE**, time **TIME**, status **VARCHAR(20)**, doctorID **INT FK → Doctor(id)**, patientID **INT FK → Patient(id)**)

6. Surgery (id **INT PK**, type **VARCHAR(100)**, date **DATE**, outcome **TEXT**, doctorID **INT FK → Doctor(id)**, patientID **INT FK → Patient(id)**)

7. Ward (id **INT PK**, wardNumber **VARCHAR(10)**, type **VARCHAR(50)**, capacity **INT**)

8. Hospitalization (id **INT PK**, checkInDate **DATE**, checkOutDate **DATE**, patientID **INT FK → Patient(id)**, wardID **INT FK → Ward(id)**)

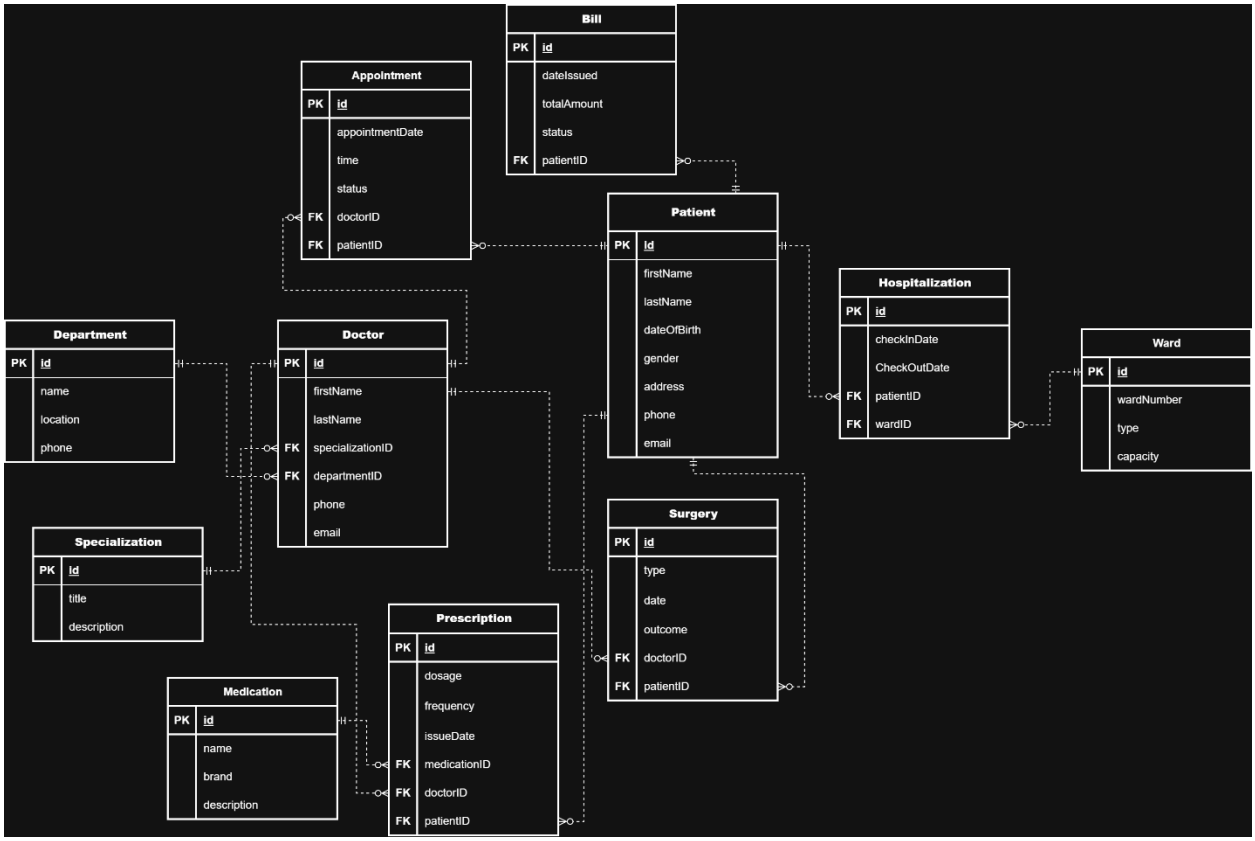
9. Medication (id **INT PK**, name **VARCHAR(100)**, brand **VARCHAR(100)**, description **TEXT**)

10. Prescription (id **INT PK**, dosage **VARCHAR(100)**, frequency **VARCHAR(100)**, issueDate **DATE**, medicationID **INT FK → Medication(id)**, doctorID **INT FK → Doctor(id)**, patientID **INT FK → Patient(id)**)

11. Bill (id **INT PK**, dateIssued **DATE**, totalAmount **DECIMAL(10,2)**, status **VARCHAR(20)**, patientID **INT FK → Patient(id)**)

Visual Assets

ER-Diagram:



Before Indexing:

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on appointment (cost=0.00..26.57 rows=1 width=34) (actual time=0.026..0.1
2	Filter: (appointmentdate = (CURRENT_DATE + '1 day'::interval))
3	Rows Removed by Filter: 1003
4	Buffers: shared hit=9
5	Planning:
6	Buffers: shared hit=4
7	Planning Time: 1.032 ms
8	Execution Time: 0.218 ms

After Indexing:

Showing rows: 1 to 8 Page No: 1 of 1

	QUERY PLAN text
1	Index Scan using idx_appointment_date on appointment (cost=0.28..8.30 rows=1 width=34) (actual
2	Index Cond: (appointmentdate = (CURRENT_DATE + '1 day'::interval))
3	Index Searches: 1
4	Buffers: shared hit=1 read=2
5	Planning:
6	Buffers: shared hit=16 read=1
7	Planning Time: 1.566 ms
8	Execution Time: 0.050 ms

Source Code

SQL Implementation (DDL):

```
-- 1. Departments
CREATE TABLE Department (
    id SERIAL PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL,
    location VARCHAR(255),
    phone VARCHAR(20) UNIQUE
);

-- 2. Specializations
CREATE TABLE Specialization (
    id SERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    description TEXT
);

-- 3. Doctors
CREATE TABLE Doctor (
    id SERIAL PRIMARY KEY,
    firstName VARCHAR(50) NOT NULL,
    lastName VARCHAR(50) NOT NULL,
    specializationID INT REFERENCES Specialization(id),
    departmentID INT REFERENCES Department(id),
    phone VARCHAR(20) UNIQUE,
    email VARCHAR(255) CHECK (email LIKE '%@%') UNIQUE
);

-- 4. Patients
CREATE TABLE Patient (
```

```
    id SERIAL PRIMARY KEY,
    firstName VARCHAR(50) NOT NULL,
    lastName VARCHAR(50) NOT NULL,
    dateOfBirth DATE CHECK (dateOfBirth <= CURRENT_DATE),
    gender CHAR(1) CHECK (gender IN ('M', 'F')),
    address TEXT,
    phone VARCHAR(20) UNIQUE,
    email VARCHAR(255) CHECK (email LIKE '%@%') UNIQUE
);

-- 5. Appointments
CREATE TABLE Appointment (
    id SERIAL PRIMARY KEY,
    appointmentDate DATE NOT NULL,
    appointmentTime TIME NOT NULL,
    status VARCHAR(20) DEFAULT 'Scheduled' CHECK (status IN ('Scheduled',
'Completed', 'Cancelled', 'No-show')),
    doctorID INT REFERENCES Doctor(id),
    patientID INT REFERENCES Patient(id)
);

-- 6. Surgeries
CREATE TABLE Surgery (
    id SERIAL PRIMARY KEY,
    surgeryType VARCHAR(100) NOT NULL,
```

```
    surgeryDate DATE NOT NULL,
    outcome TEXT,
    doctorID INT REFERENCES Doctor(id),
    patientID INT REFERENCES Patient(id)
);

-- 7. Wards
CREATE TABLE Ward (
    id SERIAL PRIMARY KEY,
    wardNumber VARCHAR(10) NOT NULL,
    wardType VARCHAR(50) CHECK (wardType IN ('General', 'VIP', 'ICU',
'Emergency', 'Maternity')),
    capacity INT CHECK (capacity > 0)
);

-- 8. Hospitalizations
CREATE TABLE Hospitalization (
    id SERIAL PRIMARY KEY,
    checkInDate DATE NOT NULL,
    checkOutDate DATE CHECK (checkOutDate >= checkInDate),
    patientID INT REFERENCES Patient(id),
    wardID INT REFERENCES Ward(id)
);

-- 9. Medications
```

```
CREATE TABLE Medication (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    brand VARCHAR(100),
    description TEXT
);

-- 10. Prescriptions
CREATE TABLE Prescription (
    id SERIAL PRIMARY KEY,
    dosage VARCHAR(100),
    frequency VARCHAR(100),
    issueDate DATE DEFAULT CURRENT_DATE,
    medicationID INT REFERENCES Medication(id),
    doctorID INT REFERENCES Doctor(id),
    patientID INT REFERENCES Patient(id)
);

-- 11. Bills
CREATE TABLE Bill (
    id SERIAL PRIMARY KEY,
    dateIssued DATE DEFAULT CURRENT_DATE,
    totalAmount DECIMAL(10,2) NOT NULL CHECK (totalAmount >= 0),
    status VARCHAR(20) DEFAULT 'Pending' CHECK (status IN ('Pending', 'Paid',
'Partially Paid', 'Refunded')),
```

```
    patientID INT REFERENCES Patient(id)
);

-- Populate the database

INSERT INTO Department (name, location, phone) VALUES ('Cardiology', 'A-Block, 2nd Floor', '87077384290');
INSERT INTO Specialization (title, description) VALUES ('Cardiologist', 'Heart health expert');

INSERT INTO Doctor (firstName, lastName, specializationID, departmentID, phone, email)
VALUES ('John', 'Smith', 1, 1, '87077384293', 'j@gmail.com');

INSERT INTO Patient (firstName, lastName, dateOfBirth, gender, address, phone)
VALUES ('Konstantin', 'Permin', '2007-05-29', 'M', 'Petropavl, Pushkina 82a', '87077384292');

INSERT INTO Appointment (appointmentDate, appointmentTime, status, doctorID, patientID) VALUES
('2026-06-01', '10:00:00', 'Scheduled', 1, 1),
('2026-03-01', '10:00:00', 'Completed', 1, 1),
('2026-01-21', '10:00:00', 'Completed', 1, 1);

INSERT INTO Bill (totalAmount, status, patientID)
```

```
VALUES (150.00, 'Pending', 1);

INSERT INTO Ward (wardNumber, wardType, capacity)
VALUES ('101A', 'General', 4);

INSERT INTO Surgery (surgeryType, surgeryDate, outcome, doctorID, patientID)
VALUES ('Appendectomy', '2026-04-10', 'Successful', 1, 1);

INSERT INTO Hospitalization (checkInDate, checkOutDate, patientID, wardID)
VALUES ('2026-04-10', '2026-04-15', 1, 1);

INSERT INTO Medication (name, brand, description) VALUES
('Aspirin', 'Bayer', 'Used to reduce pain, fever, or inflammation'),
('Amoxicillin', 'Amoxil', 'Antibiotic used to treat bacterial infections'),
('Lisinopril', 'Prinivil', 'Medication for high blood pressure');

INSERT INTO Prescription (dosage, frequency, issueDate, medicationID, doctorID,
patientID)
VALUES
('500mg', 'Once daily after meal', CURRENT_DATE, 2, 1, 1),
('100mg', 'Twice daily', CURRENT_DATE, 1, 1, 1);

-- Define INDEX  on a frequently queried column

CREATE INDEX idx_appointment_date ON Appointment(appointmentDate);
```

```
INSERT INTO Appointment (appointmentDate, appointmentTime, status, doctorID,
patientID)
SELECT
    CURRENT_DATE + (i || ' days')::interval,
    '10:00:00',
    'Scheduled',
    1,
    1
FROM generate_series(1, 1000) AS i;

ANALYZE Appointment;

-- DROP INDEX IF EXISTS idx_appointment_date;

EXPLAIN ANALYZE
SELECT * FROM Appointment
WHERE appointmentDate = CURRENT_DATE + interval '1 day';

SELECT tablename, indexname, indexdef
FROM pg_indexes
WHERE tablename = 'appointment';

-- Later Changes
```

```
ALTER TABLE appointment
DROP CONSTRAINT appointment_patientid_fkey,
ADD CONSTRAINT appointment_patientid_fkey
  FOREIGN KEY (patientID) REFERENCES patient(id) ON DELETE CASCADE;

ALTER TABLE surgery
DROP CONSTRAINT surgery_patientid_fkey,
ADD CONSTRAINT surgery_patientid_fkey
  FOREIGN KEY (patientID) REFERENCES patient(id) ON DELETE CASCADE;

ALTER TABLE hospitalization
DROP CONSTRAINT hospitalization_patientid_fkey,
ADD CONSTRAINT hospitalization_patientid_fkey
  FOREIGN KEY (patientID) REFERENCES patient(id) ON DELETE CASCADE;

ALTER TABLE prescription
DROP CONSTRAINT prescription_patientid_fkey,
ADD CONSTRAINT prescription_patientid_fkey
  FOREIGN KEY (patientID) REFERENCES patient(id) ON DELETE CASCADE;

ALTER TABLE bill
DROP CONSTRAINT bill_patientid_fkey,
ADD CONSTRAINT bill_patientid_fkey
  FOREIGN KEY (patientID) REFERENCES patient(id) ON DELETE CASCADE;
```

Application Implementation (Main.java):

The application is developed in Java 24 using JDBC. It implements full CRUD (Create, Read, Update, Delete) functionality for the Patient entity.

```
package org.example;


import java.sql.*;


public class Main {


    // ── Connection settings
──────────────────────────────────────────────────────────────────────────────

    private static final String URL      = "jdbc:postgresql://localhost:5432/Hospital";

    private static final String USER     = "postgres";

    private static final String PASSWORD = "qwerty";


    // ── Entry point
──────────────────────────────────────────────────────────────────────────────
────────────

    public static void main(String[] args) {

        try (Connection conn = DriverManager.getConnection(URL, USER, PASSWORD)) {
```

```
        if (conn != null)
            System.out.println("Connected to the database!\n");
        else {
            System.out.println("Connection attempt failed!");
            return;
        }

        // ── CREATE ──────────────
```

```
        // ── READONE
────────────
```

```
        readPatientById(conn, 1);




    } catch (SQLException e) {

        e.printStackTrace();

    }

}


// ── CREATE
──────────────
```

```
String sql = "INSERT INTO patient (firstname, lastname, dateofbirth, gender, address, phone, email) "
        + "VALUES (?, ?, ?, ?, ?, ?, ?)";


try (PreparedStatement pstmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
    pstmt.setString(1, firstname);
    pstmt.setString(2, lastname);
    pstmt.setDate(3, java.sql.Date.valueOf(dateofbirth));        pstmt.setString(4, gender);
    pstmt.setString(5, address);
    pstmt.setString(6, phone);
    pstmt.setString(7, email);




    int rowsAffected = pstmt.executeUpdate();
    if (rowsAffected > 0) {
        try (ResultSet generatedKeys = pstmt.getGeneratedKeys()){
            if (generatedKeys.next()) {
```

```
                long id = generatedKeys.getLong(1);

                System.out.println("Inserted successfully! New Patient ID:"  + id);
}

            }

        }

    }

  }


  // ── READ (all) ──────────────────────────────────────────────────────────────────────────────


  public static void readAllPatients(Connection conn) throws SQLException {

    String sql = "SELECT id, firstname, lastname, dateofbirth, gender, address, phone, email
FROM patient";


    try (Statement stmt = conn.createStatement();

        ResultSet rs   = stmt.executeQuery(sql)) {


        int count = 0;

        while (rs.next()) {
```

```
            printPatient(rs);
            count++;
        }
        System.out.println("Total rows: " + count);
    }
}

// ── READ (single) ──────────────────────────────────────────────────────────────────────────────
public static void readPatientById(Connection conn, Integer id) throws SQLException {

    String sql = "SELECT id, firstname, lastname, dateofbirth, gender, address, phone, email "
            + "FROM patient WHERE id = ?";

    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, id);

        try (ResultSet rs = pstmt.executeQuery()) {
```

```
            if (rs.next())
                printPatient(rs);
            else
                System.out.println("No Patient found with ID: " + id);
        }
    }
}
// ── READ (single) ──────────────────────────────────────────────────────────────────────────────
public static void readPatientByLastName(Connection conn, String lastname) throws SQLException {

    String sql = "SELECT id, firstname, lastname, dateofbirth, gender, address, phone, email "
            + "FROM patient WHERE lastname = ?";

    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, lastname);

        try (ResultSet rs = pstmt.executeQuery()) {
```

```
            if (rs.next())
                printPatient(rs);
            else
                System.out.println("No Patient found with lastname: " + lastname);
        }
    }
}

// ── UPDATE ──────────────
```

```
                                String newemail) throws SQLException {


        String sql = "UPDATE patient "
                + "SET firstname = ?, lastname = ?, dateofbirth = ?, gender = ?, address = ?, phone
= ?, email = ? "
                + "WHERE id = ?";


        try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, newfirstname);
            pstmt.setString(2, newlastname);
            pstmt.setDate(3, java.sql.Date.valueOf(newdateofbirth));            pstmt.setString(4,
newgender);
            pstmt.setString(5, newaddress);
            pstmt.setString(6, newphone);
            pstmt.setString(7, newemail);
            pstmt.setInt(8, id);


            int rowsAffected = pstmt.executeUpdate();
            if (rowsAffected > 0)
                System.out.println("Updated patientID: " + id);
```

```
        else
            System.out.println("No Patient found with ID: " + id);
    }
}

// ── DELETE ──────────────
```

```
        }
    }

    // ── Helper ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

    private static void printPatient(ResultSet rs) throws SQLException {
        System.out.printf("ID: %-10s | firstname: %-30s | lastname: %-20s |dateofbirth: %20s |gender: %-20s | address: %-20s | phone: %-20s | email: %s%n",
                rs.getInt("id"),
                rs.getString("firstname"),
                rs.getString("lastname"),
                rs.getString("dateofbirth"),
                rs.getString("gender"),
                rs.getString("address"),
                rs.getString("phone"),
                rs.getString("email"));
    }
}
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.1.1\lib\idea_rt.jar=61373" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=
Connected to the database!

=== CREATE ===
Inserted successfully! New Patient ID:11
=== READ ===
ID: 2          | firstname: Sabina                        | lastname: Sadykova            |dateofbirth:          2006-10-24 |gender: F                    | address: Petropa
ID: 3          | firstname: Sabinas                       | lastname: Sadykovas           |dateofbirth:          2006-10-24 |gender: F                    | address: Petropa
ID: 5          | firstname: Sabinass                      | lastname: Sadykovass          |dateofbirth:          2006-10-24 |gender: F                    | address: Petropa
ID: 7          | firstname: SSabinass                     | lastname: SSadykovass         |dateofbirth:          2006-10-24 |gender: F                    | address: Petropa
ID: 8          | firstname: SSabinaass                    | lastname: SSadykoovass        |dateofbirth:          2006-10-24 |gender: F                    | address: Petropa
ID: 10         | firstname: SSaabinaass                   | lastname: SSaadykoovass       |dateofbirth:          2006-10-24 |gender: F                    | address: Petropa
ID: 11         | firstname: SSasabinaass                  | lastname: SSaasdykoovass      |dateofbirth:          2006-10-24 |gender: F                    | address: Petropa
Total rows: 7
=== READ ===
ID: 2          | firstname: Sabina                        | lastname: Sadykova            |dateofbirth:          2006-10-24 |gender: F                    | address: Petropa
No Patient found with lastname: Permin
=== UPDATE ===
Updated patientID: 2
=== DELETE ===
Deleted PatientID: 3
No Patient found with ID: 3

Process finished with exit code 0
```

Git Repository

The complete source code, including the database scripts and Java application, is available at:

URL: [Bu88lleGum/Hospital-Management-System]

Conclusion

In this project, I successfully developed a functional Hospital Management System by integrating PostgreSQL and MongoDB. I learned how to bridge the gap between Java and SQL using JDBC, handle data type conversions (like java.sql.Date), and enforce constraints across different database paradigms. The implementation of "Cascading Deletes" and "JSON Schemas" ensured that the system remains stable and prevents data orphans.